

Soluzione

Soluzione commentata — Pomodoro Focus Timer (Python + SOLID)

`timer.py` — M1 (SRP + DIP)

```
from __future__ import annotations
from typing import Protocol, Callable
import time

class ITickProvider(Protocol):
    """Astrae l'attesa di un 'tick' (normalmente 1 secondo)."""
    def tick(self) → None: ...

class RealTickProvider:
    """Implementazione reale: attende 1 secondo tra un tick e l'altro."""
    def tick(self) → None:
        time.sleep(1)

class TimerService:
    """
    Responsabilità: eseguire un countdown e chiamare due callback:
    - on_tick(int seconds_left): ad ogni secondo rimanente
    - on_completed(): al termine
    DIP: dipende da ITickProvider, sostituibile nei test.
    """
    def __init__(self, tick_provider: ITickProvider | None = None) → None:
        self._tick = tick_provider or RealTickProvider()

    def countdown(
        self,
        seconds: int,
        *,
        on_tick: Callable[[int], None],
        on_completed: Callable[[], None]
    ) → None:
        """
```

Regole:

- seconds < 0 → ValueError

- per s = seconds..1:

 - chiama on_tick(s)

 - attendi 1 tick

- poi chiama on_completed()

Nota: se seconds == 0, non chiama on_tick e va diretto a on_complete

d.

```
"""
```

```
if seconds < 0:
```

```
    raise ValueError("seconds deve essere >= 0")
```

```
for s in range(seconds, 0, -1):
```

```
    on_tick(s)
```

```
    self._tick.tick()
```

```
on_completed()
```

sessions.py — M3 (OCP)

```
from __future__ import annotations
```

```
from typing import Protocol, Tuple
```

```
class ISessionStrategy(Protocol):
```

```
    def get_durations(self) → Tuple[int, int]:
```

```
        """Ritorna (focus_seconds, break_seconds)."""
```

```
    ...
```

```
class Classic25_5:
```

```
    def get_durations(self) → Tuple[int, int]:
```

```
        return 25 * 60, 5 * 60
```

```
class Deep50_10:
```

```
    def get_durations(self) → Tuple[int, int]:
```

```
        return 50 * 60, 10 * 60
```

```
class Custom:
```

```

"""
Strategia personalizzabile. Valida input (> 0).
Mantiene OCP: puoi aggiungere nuove strategie senza toccare Pomodor
o.
"""
def __init__(self, focus_sec: int, break_sec: int) → None:
    # Validazione semplice e chiara (fa fallire i test se assente)
    if not isinstance(focus_sec, int) or not isinstance(break_sec, int):
        raise ValueError("focus_sec e break_sec devono essere interi")
    if focus_sec <= 0 or break_sec <= 0:
        raise ValueError("focus_sec e break_sec devono essere > 0")
    self.focus_sec = focus_sec
    self.break_sec = break_sec

def get_durations(self) → Tuple[int, int]:
    return self.focus_sec, self.break_sec

```

notify.py — (ISP + DIP)

```

from __future__ import annotations
from typing import Protocol

class INotifier(Protocol):
    """Interfaccia piccola e mirata (ISP)."""
    def notify(self, message: str) → None: ...

class ConsoleNotifier:
    """Implementazione console-based; separata dalla logica core."""
    def notify(self, message: str) → None:
        print(message)

class BeepNotifier:
    """Beep testuale (non garantito su tutte le console, ma innocuo)."""
    def notify(self, message: str) → None:
        print("\a" + message) # \a = bell (spesso silenziato)

```

persistence.py — M5 (SRP + DIP)

```
from __future__ import annotations
from typing import Protocol, Literal
from dataclasses import dataclass
from datetime import datetime
import csv
import os

@dataclass
class SessionLog:
    started_at: datetime
    kind: Literal["focus", "break"]
    seconds: int

class ISessionRepository(Protocol):
    """Astrae la persistenza: Pomodoro non conosce il 'come'."""
    def save(self, log: SessionLog) → None: ...

class CsvSessionRepository:
    """
    SRP: si occupa solo di persistenza su CSV.
    DIP: usata tramite interfaccia ISessionRepository.
    """
    def __init__(self, path: str = "sessions.csv") → None:
        self.path = path
        # Se il file non esiste, crea header
        if not os.path.exists(self.path):
            with open(self.path, "w", newline="", encoding="utf-8") as f:
                writer = csv.writer(f)
                writer.writerow(["started_at_iso", "kind", "seconds"])

    def save(self, log: SessionLog) → None:
        """
        Appende una riga con:
        - started_at_iso = log.started_at.isoformat()
        - kind           = "focus" | "break"
        - seconds        = int
        """
```

```

"""
if log.kind not in ("focus", "break"):
    raise ValueError("kind deve essere 'focus' o 'break'")
if log.seconds < 0:
    raise ValueError("seconds deve essere >= 0")

with open(self.path, "a", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow([log.started_at.isoformat(), log.kind, str(log.seconds)])

```

core.py — M4 + M5 (Observer-like + Orchestrazione)

```

from __future__ import annotations
from dataclasses import dataclass, field
from typing import Callable, Optional
from datetime import datetime

from .timer import TimerService
from .notify import INotifier, ConsoleNotifier
from .sessions import ISessionStrategy
from .persistence import ISessionRepository, CsvSessionRepository, SessionLog

@dataclass
class Pomodoro:
    """
    Orchestratore:
    - legge le durate dalla strategia
    - esegue il countdown focus → break
    - notifica a fine fase
    - salva i log (repo)
    - espone un callback-evento on_focus_completed
    """
    strategy: ISessionStrategy
    notifier: INotifier = field(default_factory=ConsoleNotifier)

```

```

timer: TimerService = field(default_factory=TimerService)
repo: ISessionRepository = field(default_factory=CsvSessionRepository)

# Callback opzionale (Observer-like) chiamata al termine del focus.
on_focus_completed: Optional[Callable[[], None]] = None

def run(self) → None:
    focus_sec, break_sec = self.strategy.get_durations()

    # ----- FOCUS -----
    focus_start = datetime.now()

    def _focus_completed():
        # Notifica
        self.notifier.notify("Focus finito!")
        # Evento opzionale
        if self.on_focus_completed:
            self.on_focus_completed()
        # Persistenza
        self.repo.save(SessionLog(started_at=focus_start, kind="focus", seconds=focus_sec))

    # on_tick per il focus: qui non facciamo I/O (SRP); i test non richiedono output.
    self.timer.countdown(
        focus_sec,
        on_tick=lambda s: None,
        on_completed=_focus_completed
    )

    # ----- BREAK -----
    break_start = datetime.now()

    def _break_completed():
        self.notifier.notify("Break finito!")
        self.repo.save(SessionLog(started_at=break_start, kind="break", seconds=break_sec))

```

```
self.timer.countdown(  
    break_sec,  
    on_tick=lambda s: None,  
    on_completed=_break_completed  
)
```

runner.py — (facoltativo, invariato)

```
from __future__ import annotations  
from .sessions import Custom, Classic25_5, Deep50_10, ISessionStrategy  
from .core import Pomodoro  
  
def choose_strategy() → ISessionStrategy:  
    print("Scegli strategia: [1] Classic 25/5 [2] Deep 50/10 [3] Custom breve  
demo (10s/5s)")  
    choice = input("Scelta (1/2/3): ").strip()  
    if choice == "1":  
        return Classic25_5()  
    if choice == "2":  
        return Deep50_10()  
    return Custom(10, 5) # default: durate brevi per demo  
  
def main() → None:  
    strat = choose_strategy()  
    pomodoro = Pomodoro(strat)  
    pomodoro.run()  
  
if __name__ == "__main__":  
    main()
```

Note sui test

Dopo aver incollato le implementazioni:

- In `tests/test_timer.py`, rimuovi il blocco con `with pytest.raises(NotImplementedError)` e attiva le asserzioni suggerite nei commenti.
- Fai lo stesso in `tests/test_pomodoro.py` e `tests/test_repository.py`.

- I test su `Custom` dovrebbero ora passare grazie alla validazione.

Esegui:

```
pytest
```